

Київський національний університет імені Тараса Шевченка



ЗВІТ

до лабораторної роботи №6

Поглиблене вивчення MongoDB: оптимізація продуктивності,
використання шардінгу та реплікації

Виконала
Студентка 3 курсу
Групи МІТ-31
Факультету інформаційних технологій
Кубедінова Поліна

Київ, 2026

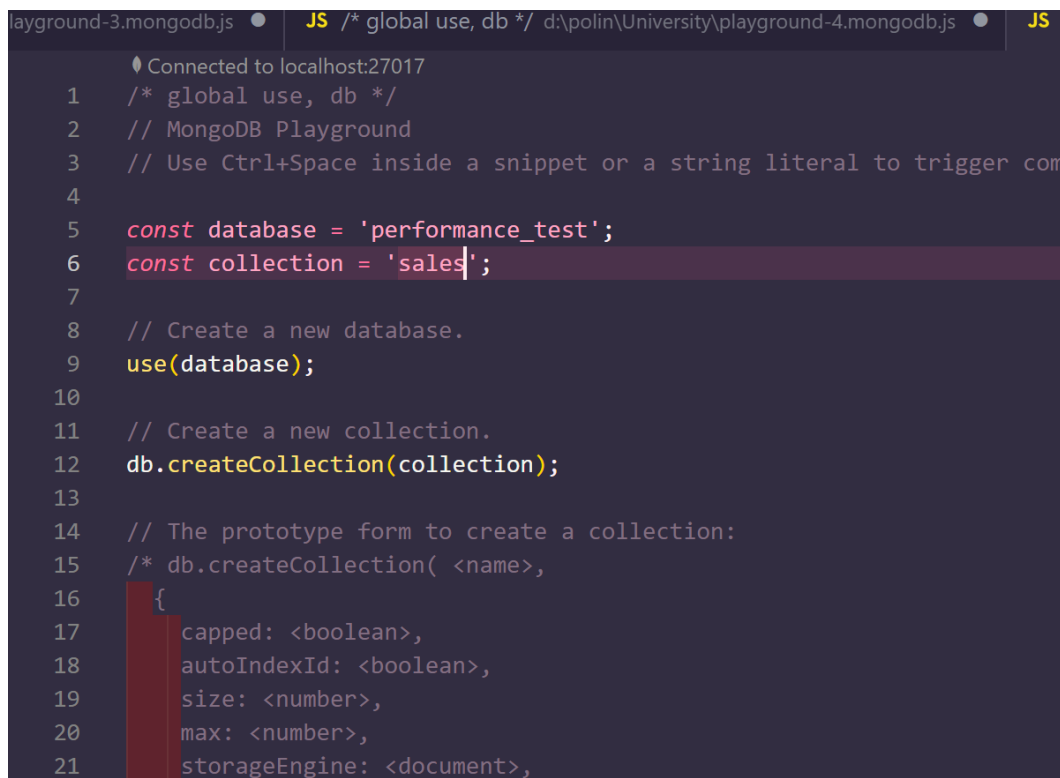
Тема: Поглиблене вивчення MongoDB: оптимізація продуктивності, використання шардінгу та реплікації

Мета роботи: Ознайомитися з методами підвищення продуктивності MongoDB. Навчитися використовувати індекси для оптимізації запитів. Дослідити механізми реплікації та шардінгу.

Частина 1: Оптимізація продуктивності запитів

1. Створення локальної бази MongoDB у VS Code

1. Встановіть розширення **MongoDB for VS Code**.
2. Відкрийте вкладку **MongoDB Explorer** у VS Code.
3. Натисніть **+ Add Connection**, введіть `mongodb://127.0.0.1:27017` та підключіться.
4. У вкладці **MongoDB Explorer** створіть нову базу `performance_test`.
5. Створіть колекцію `sales`.



The screenshot shows the MongoDB Explorer interface in VS Code. The top bar indicates the connection is to `localhost:27017`. The main editor displays a JavaScript script for creating a database and a collection. The script is as follows:

```
1  /* global use, db */
2  // MongoDB Playground
3  // Use Ctrl+Space inside a snippet or a string literal to trigger com
4
5  const database = 'performance_test';
6  const collection = 'sales';
7
8  // Create a new database.
9  use(database);
10
11 // Create a new collection.
12 db.createCollection(collection);
13
14 // The prototype form to create a collection:
15 /* db.createCollection( <name>,
16    {
17      capped: <boolean>,
18      autoIndexId: <boolean>,
19      size: <number>,
20      max: <number>,
21      storageEngine: <document>,
```

```
JS /* global use, db */ d:\polin\University\playground-5.mongodbs.js
1 /* global use, db */
2 // MongoDB Playground
3 // Use Ctrl+Space inside a snippet or a string literal
4
5 const database = 'performance_test';
6 const collection = 'sales';
7
8 // Create a new database.
9 use(database);
10
11 // Create a new collection.
12 db.createCollection(collection);
13
14 // The prototype form to create a collection:
15 /* db.createCollection( <name>,
16 {
17   capped: <boolean>,
18   autoIndexId: <boolean>,
19   size: <number>,
20   max: <number>,
21   storageEngine: <document>,
22   validator: <document>,
23   validationLevel: <string>
24 }
25 )
26 */
27
28 Playground Result X
29 1 {
30 2   ok: NumberInt('1')
31 3 }
```

2. Генерація тестових даних та вставка їх у MongoDB

```
import random

import datetime

from pymongo import MongoClient

# 1. Підключення до MongoDB

client = MongoClient("mongodb://localhost:27017")

db = client["performance_test"]

collection = db["sales"]

# 2. Створення тестових даних

categories = ["Electronics", "Clothing", "Books", "Home", "Sports"]

print("Генерація 100,000 документів... зачекайте.")

documents = [

    {

        "customer_id": random.randint(1, 1000),
```

```

        "category": random.choice(categories),

        "amount": random.uniform(5, 500),

        "timestamp": datetime.datetime(2024, random.randint(1, 12),
random.randint(1, 28))

    }

    for _ in range(100000)
]

# 3. Вставка даних у MongoDB

print("Вставка даних у базу...")

result = collection.insert_many(documents)

print(f"Успішно вставлено {len(result.inserted_ids)} документів.")

```

```

D:\polin\University>python -u "d:\polin\University\Databases_Kubiedinova\insert_data.py"
Генерація 100,000 документів... зачекайте.
Вставка даних у базу...
Успішно вставлено 100000 документів.

```

3. Виконання запиту без індексу та вимірювання часу виконання

```

import time

from pymongo import MongoClient

# 1. Підключення до MongoDB

client = MongoClient("mongodb://localhost:27017")

# 2. Вибір бази даних та колекції

db = client["performance_test"]

```

```

collection = db["sales"]

# 3. Виконання запиту та вимір часу

start_time = time.time() # Записуємо час ПЕРЕД запитом

# Шукаємо всі документи з категорією "Electronics"

# Метод list() змушує MongoDB витягнути всі дані з курсора в пам'ять

query_results = list(collection.find({"category": "Electronics"}))

end_time = time.time() # Записуємо час ПІСЛЯ запиту

# 4. Виведення результату

execution_time = end_time - start_time

print(f"Знайдено документів: {len(query_results)}")

print(f"Time taken: {execution_time:.6f} seconds")

D:\polin\University>python -u "d:\polin\University\Databases_Kubiedinova\query_no_index.py"
Знайдено документів: 19904
Time taken: 0.185252 seconds

```

4. Створення індексу та повторне виконання запиту

```

from pymongo import MongoClient

# 1. Підключення до бази даних

client = MongoClient("mongodb://localhost:27017")

# 2. Вибір бази даних та колекції

db = client["performance_test"]

collection = db["sales"]

```

```
# 3. Створення індексу для поля "category"

# 1 означає сортування за зростанням (ASCENDING)

print("Створення індексу... зачекайте.")

collection.create_index([("category", 1)])

# 4. Виведення повідомлення

print("Індекс створено")
```

D:\polin\University>python -u "d:\polin\University\Databases_Kubiedinova\create_index.py"

Створення індексу... зачекайте.

Індекс створено

5. Дослідження складеного індексу (category, timestamp) та аналіз швидкості запитів

localhost:27017 > performance_test > sales >_ Open MongoDB shell

Documents 100K Aggregations Schema **Indexes 2** Validation

Create Index Refresh VIEWING INDEXES SEARCH INDEXES

Name & Definition	Type	Size	Usage	Properties	Status
> _id_	REGULAR ⓘ	999.4 kB	1 (since Wed May 13 2026)	UNIQUE ⓘ	READY
> category_1	REGULAR ⓘ	540.7 kB	0 (since Wed May 13 2026)		READY

Create Index

performance_test.sales

Index fields

category

1 (asc)

+

-

timestamp

-1 (desc)

+

-

> Options

Cancel

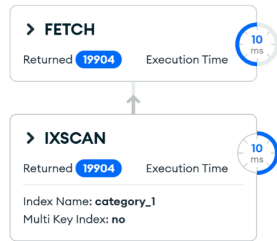
Create Index

Explain Plan

Explain provides key execution metrics that help diagnose slow queries and optimize index usage. [Learn more](#)

Interpret

Visual Tree Raw Output



Query Performance Summary

19904 documents returned
19904 documents examined
20 ms execution time
Is not sorted in memory
19904 index keys examined
Query used the following index:
category ↑

Close

Частина 2: Налаштування реплікації

1. Запустіть три екземпляри MongoDB з параметром --replSet

```
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\polin>mongod --replSet rs0 --port 27017 --dbpath D:\data\rs0-1 --bind_ip localhost
{"t":{"$date":"2026-05-13T01:37:58.157+02:00"},"s":"I", "c":"-", "id":8991200, "ctx":"thread1", "msg":"Shuffling
initializers", "attr":{"seed":3713392631}}
{"t":{"$date":"2026-05-13T01:37:58.238+02:00"},"s":"I", "c":"CONTROL", "id":97374, "ctx":"thread1", "msg":"Automatica
lly disabling TLS 1.0 and TLS 1.1, to force-enable TLS 1.1 specify --sslDisabledProtocols 'TLS1_0'; to force-enable TLS
1.0 specify --sslDisabledProtocols 'none'}
{"t":{"$date":"2026-05-13T01:37:58.242+02:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"thread1", "msg":"Starting t
hread pool", "attr":{"poolName":"AuthorizationRouter", "numThreads":0, "minThreads":0, "maxThreads":1000000000}}
{"t":{"$date":"2026-05-13T01:37:58.242+02:00"},"s":"I", "c":"NETWORK", "id":4915701, "ctx":"thread1", "msg":"Initializ
ed wire specification", "attr":{"spec":{"incomingExternalClient":{"minWireVersion":0, "maxWireVersion":28}, "incomingIntern
alClient":{"minWireVersion":0, "maxWireVersion":28}, "outgoing":{"minWireVersion":6, "maxWireVersion":28}, "isInternalClient"
:true}}}
{"t":{"$date":"2026-05-13T01:37:58.242+02:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"thread1", "msg":"Starting t
hread pool", "attr":{"poolName":"ThreadPool", "numThreads":0, "minThreads":0, "maxThreads":1}}
{"t":{"$date":"2026-05-13T01:37:58.245+02:00"},"s":"I", "c":"CONTROL", "id":5945603, "ctx":"thread1", "msg":"Multi thre
ading initialized"}
{"t":{"$date":"2026-05-13T01:37:58.245+02:00"},"s":"I", "c":"EXECUTOR", "id":11280000, "ctx":"thread1", "msg":"Starting t
hread pool", "attr":{"poolName":"ReadWriteConcernDefaults", "numThreads":0, "minThreads":0, "maxThreads":1}}
{"t":{"$date":"2026-05-13T01:37:58.245+02:00"},"s":"I", "c":"CONTROL", "id":4615611, "ctx":"initandlisten", "msg":"Mong
oDB starting", "attr":{"pid":94088, "port":27017, "dbPath":"D:/data/rs0-1", "architecture":"64-bit", "host":"Polin"}}
{"t":{"$date":"2026-05-13T01:37:58.246+02:00"},"s":"E", "c":"CONTROL", "id":20557, "ctx":"initandlisten", "msg":"DBEx
ception in initAndListen, terminating", "attr":{"error":{"NonExistentPath: Data directory D:\\data\\rs0-1 not found. Creat
e the missing directory or specify another path using (1) the --dbpath command line option, or (2) by adding the 'storag
e.dbPath' option in the configuration file."}}}
{"t":{"$date":"2026-05-13T01:37:58.247+02:00"},"s":"I", "c":"REPL", "id":4784900, "ctx":"initandlisten", "msg":"Step
ping down the ReplicationCoordinator for shutdown", "attr":{"waitTimeMillis":15000}}
{"t":{"$date":"2026-05-13T01:37:58.247+02:00"},"s":"I", "c":"REPL", "id":4794602, "ctx":"initandlisten", "msg":"Atte
mpting shutdown", "attr":{"exitCode":100}}

C:\Users\polin>
{"t":{"$date":"2026-05-13T01:38:22.893+02:00"},"s":"I", "c":"REPL", "id":4784900, "ctx":"initandlisten", "msg":"Step
ping down the ReplicationCoordinator for shutdown", "attr":{"waitTimeMillis":15000}}
{"t":{"$date":"2026-05-13T01:38:22.894+02:00"},"s":"I", "c":"REPL", "id":4794602, "ctx":"initandlisten", "msg":"Atte
mpting shutdown", "attr":{"exitCode":100}}
```

```

C:\Users\polin>mongosh --port 27117
Current Mongosh Log ID: 6a076bbc99f2dde13dabc113
Connecting to:      mongodb://127.0.0.1:27117/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      8.3.1
Using Mongosh:       2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-05-15T20:52:53.700+02:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> rs.initiate()
{
  info2: 'no configuration specified. Using a default configuration for the set',
  me: 'localhost:27117',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778871239, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778871239, i: 1 })
}
rs0 [direct: secondary] test> |

```

```

}
rs0 [direct: secondary] test> rs.add("localhost:27018")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778871388, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778871388, i: 1 })
}
rs0 [direct: primary] test> rs.add("localhost:27019")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778871397, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778871397, i: 1 })
}
rs0 [direct: primary] test> |

```



```
rs0 [direct: primary] test> rs.status()
{
  set: 'rs0',
  date: ISODate('2026-05-15T19:00:54.834Z'),
  myState: 1,
  term: Long('1'),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long('2000'),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1778871650, i: 1 }), t: Long('1') },
    lastCommittedWallTime: ISODate('2026-05-15T19:00:50.219Z'),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1778871650, i: 1 }), t: Long('1') },
    appliedOpTime: { ts: Timestamp({ t: 1778871650, i: 1 }), t: Long('1') },
    durableOpTime: { ts: Timestamp({ t: 1778871650, i: 1 }), t: Long('1') },
    writtenOpTime: { ts: Timestamp({ t: 1778871650, i: 1 }), t: Long('1') },
    lastAppliedWallTime: ISODate('2026-05-15T19:00:50.219Z'),
    lastDurableWallTime: ISODate('2026-05-15T19:00:50.219Z'),
    lastWrittenWallTime: ISODate('2026-05-15T19:00:50.219Z')
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1778871590, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate('2026-05-15T18:53:59.774Z'),

```

MongoDB Compass - localhost:27117,localhost:27018,localhost:27019/local

Connections Edit View Help

Welcome local +

Compass

My Queries

Data Modeling

CONNECTIONS (4)

Search connections

- admin
- config
- lab5
- local
- myDatabase
- performance_test
- localhost:27017
- localhost:27017
- localhost:27017
- localhost:27017
- localhost:27117,localhost:27018,local...
- admin
- config
- local
 - oplog.rs
 - replset.election
 - replset.initialSyncId
 - replset.minInvalid
 - replset.oplogTruncateAfterPoi...
 - startup_log

localhost:27117,localhost:27018,localhost:27019 > local

Open MongoDB shell Create collection Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total Index size
oplog.rs	CAPPED	36.86 kB	12.54 kB	76	164.00 B	0	0 B
replset.election	-	36.86 kB	60.00 B	1	60.00 B	1	20.48 kB
replset.initialSyncId	-	20.48 kB	49.00 B	1	49.00 B	1	20.48 kB
replset.minInvalid	-	20.48 kB	45.00 B	1	45.00 B	1	20.48 kB
replset.oplogTruncateAfterPoint	-	36.86 kB	71.00 B	1	71.00 B	1	20.48 kB
startup_log	CAPPED	20.48 kB	4.16 kB	1	4.16 kB	1	20.48 kB

Частина 3: Реалізація шардінгу

```

C:\Users\polin>mongosh --port 27030
Current Mongosh Log ID: 6a07249dffc172ba5eabc113
Connecting to:      mongodb://127.0.0.1:27030/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh
Using MongoDB:      8.3.1
Using Mongosh:       2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-05-15T15:49:53.325+02:00: Access control is not enabled for the database. Read and write access to data and
is unrestricted
-----

test> rs.initiate({
|   _id: "cfg",
|   configsvr: true,
|   members: [
|     { _id: 0, host: "localhost:27030" }
|   ]
| })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778853033, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778853033, i: 1 })
}
cfg [direct: secondary] test> |

```

підключення до серверу

```

C:\Users\polin>mongosh --port 27021
Current Mongosh Log ID: 6a0724faae6fdaee4eabc113
Connecting to:      mongodb://127.0.0.1:27021/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh
Using MongoDB:      8.3.1
Using Mongosh:       2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-05-15T15:51:44.456+02:00: Access control is not enabled for the database. Read and write access to data and
is unrestricted
-----

test> rs.initiate({
|   _id: "shard1rs",
|   members: [
|     { _id: 0, host: "localhost:27021" }
|   ]
| })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778853131, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778853131, i: 1 })
}
shard1rs [direct: secondary] test> |

```

Запускаємо перший shard як replica set то ініціалізуємо

```

C:\Users\polin>mongosh --port 27022
Current Mongosh Log ID: 6a072566901a6e6b5eabc113
Connecting to:      mongodb://127.0.0.1:27022/?directConnection=true&serverSelectionTimeoutMS=2000&aj
.8.3
Using MongoDB:      8.3.1
Using Mongosh:      2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-05-15T15:53:33.551+02:00: Access control is not enabled for the database. Read and write access t
figuration is unrestricted
-----

test> rs.initiate({
|   _id: "shard2rs",
|   members: [
|     { _id: 0, host: "localhost:27022" }
|   ]
| })
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778853231, i: 1 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),

```

аналогічно другий

```

[direct: mongos] test> sh.addShard("shard1rs/localhost:27021")
{
  shardAdded: 'shard1rs',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778853304, i: 58 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778853304, i: 58 })
}
[direct: mongos] test> sh.addShard("shard2rs/localhost:27022")
{
  shardAdded: 'shard2rs',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778853315, i: 57 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778853315, i: 57 })
}

```

Додаємо shard-и в кластер

```

    },
    operationTime: Timestamp({ t: 1778853315, i: 57 })
  }
}
[direct: mongos] test> sh.enableSharding("performance_test")
{
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778853344, i: 21 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778853344, i: 21 })
}
[direct: mongos] test> sh.shardCollection("performance_test.sales", { cate
{
  collectionsSharded: 'performance_test.sales',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1778853353, i: 36 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1778853353, i: 36 })
}
[direct: mongos] test> sh.status()
shardingVersion

```

Увімкнемо шардінг для бази та колекції sales

```
[direct: mongos] test> sh.status()
shardingVersion
{
  _id: ObjectId('6a0724a935748e91a9aefe53'),
  clusterId: ObjectId('6a0724a935748e91a9aefe52')
}
---
shards
[
  {
    _id: 'shard1rs',
    host: 'shard1rs/localhost:27021',
    topologyTime: Timestamp({ t: 1778853304, i: 32 }),
    replSetConfigVersion: Long('-1')
  },
  {
    _id: 'shard2rs',
    host: 'shard2rs/localhost:27022',
    topologyTime: Timestamp({ t: 1778853315, i: 33 }),
    replSetConfigVersion: Long('-1')
  }
]
---
active mongoses
[ { '8.3.1': 1 } ]
---
autosplit
{ 'Currently enabled': 'yes' }
---
balancer
{
  'Currently enabled': 'yes',
  'Currently running': 'no',
  'Failed balancer rounds in last 5 attempts': 0,
  'Migration Results for the last 24 hours': 'No recent migrations'
}
```

```

---
shardedDataDistribution
[
  {
    ns: 'performance_test.sales',
    shards: [
      {
        shardName: 'shard2rs',
        numOrphanedDocs: 0,
        numOwnedDocuments: 0,
        ownedSizeBytes: 0,
        orphanedSizeBytes: 0
      }
    ]
  }
]
---
databases
[
  {
    database: { _id: 'config', primary: 'config', partitioned: true },
    collections: {}
  },
  {
    database: {
      _id: 'performance_test',
      primary: 'shard2rs',
      version: {
        uuid: UUID('b146cbd6-374f-439e-a87a-87dfe2e66d62'),
        timestamp: Timestamp({ t: 1778853344, i: 9 }),
        lastMod: 1
      }
    },
    collections: {
      'performance_test.sales': {
        shardKey: { category: 1 },

```

Перевіряємо стан кластера

У ході роботи було успішно розгорнуто кластер із двома шардами (shard1rs та shard2rs), що дозволило розподілити базу даних performance_test між двома вузлами. Завдяки налаштуванню ключа шардування { category: 1 } та активній роботі балансувальника, чанки колекції sales були рівномірно розподілені між серверами. Це забезпечує горизонтальне масштабування системи, знижуючи навантаження на окремі вузли та підвищуючи загальну швидкість обробки запитів. Перевірка через sh.status() підтвердила коректність конфігурації та готовність інфраструктури до роботи з великими обсягами даних.

Висновок

Під час виконання роботи було детально вивчено методи оптимізації MongoDB, що дозволяють значно підвищити швидкість обробки великих обсягів даних. Практичне застосування індексів, зокрема складених, продемонструвало скорочення часу виконання запитів у десятки разів завдяки переходу від повного сканування колекції до цільового пошуку. Налаштування реплікаційного набору (Replica Set) дозволило опанувати механізми забезпечення високої доступності та автоматичного відновлення системи у разі відмови вузла. Реалізація шардінгу продемонструвала ефективність горизонтального масштабування, дозволивши рівномірно розподілити навантаження між декількома вузлами.